

同济大学计算机系

计算机组成原理实验报告



学 号 2454307

姓 名 汤皓宇

专 业 信息安全

授课老师 郭玉臣

一、实验内容

无符号乘法器、有符号乘法器

二、硬件逻辑图

无要求

三、模块建模

1. 有符号乘法器

这个 MULTU 模块实现了一个 64 位结果的 32 位无符号流水线乘法器，它通过硬件逻辑模拟了手动乘法中“移位并相加”的过程。具体实现上，模块采用了一种六级深度的流水线加法树结构：在第一级流水线中，它根据操作数 b 的每一位状态，将操作数 a 进行对应的左移，生成 32 个部分积；随后在接下来的五级流水线中，利用树状求和逻辑，每一层都将上一层相邻的两个部分积两两相加，使数据项数量从 32 个逐级减半至 16、8、4、2 个，最终在最后一级完成最后的加法并输出结果。这种逻辑结构在每个时钟周期都能输入新的数据，通过增加时钟延迟换取了更高的运算吞吐量，能够平滑地处理高速连续的乘法运算需求。

```
module MULTU (  
    input  clk,  
    input  reset,  
    input  [31:0] a,  
    input  [31:0] b,  
    output [63:0] z  
);  
    reg [63:0] pipe1 [31:0];  
    reg [63:0] pipe2 [15:0];  
    reg [63:0] pipe3 [7:0];  
    reg [63:0] pipe4 [3:0];  
    reg [63:0] pipe5 [1:0];  
    reg [63:0] pipe6;  
  
    integer i;  
  
    always @(posedge clk) begin  
        if (reset) begin  
            for (i = 0; i < 32; i = i + 1) pipe1[i] <= 64'b0;  
            for (i = 0; i < 16; i = i + 1) pipe2[i] <= 64'b0;  
            for (i = 0; i < 8; i = i + 1) pipe3[i] <= 64'b0;  
            for (i = 0; i < 4; i = i + 1) pipe4[i] <= 64'b0;  
            for (i = 0; i < 2; i = i + 1) pipe5[i] <= 64'b0;  
            pipe6 <= 64'b0;  
        end  
    end  
endmodule
```

```

        end else begin
            for (i = 0; i < 32; i = i + 1) pipe1[i] <= b[i] ? (a << i) : 64'b0;
            for (i = 0; i < 16; i = i + 1) pipe2[i] <= pipe1[2*i] + pipe1[2*i+1];
            for (i = 0; i < 8; i = i + 1) pipe3[i] <= pipe2[2*i] + pipe2[2*i+1];
            for (i = 0; i < 4; i = i + 1) pipe4[i] <= pipe3[2*i] + pipe3[2*i+1];
            for (i = 0; i < 2; i = i + 1) pipe5[i] <= pipe4[2*i] + pipe4[2*i+1];
            pipe6 <= pipe5[0] + pipe5[1];
        end
    end

    assign z = pipe6;

endmodule

```

2. 无符号乘法器

这个 MULT 模块实现了一个基于流水线结构的 32 位有符号整数乘法器，其核心逻辑在于将复杂的乘法运算拆解为符号位扩展的部分积生成与树状加法阶段。在实现方式上，它首先将输入数据声明为 signed 类型，并在第一级流水线中根据乘数 b 的每一位生成 32 个部分积：前 31 位按位权进行左移并做 64 位符号位扩展，而针对最高符号位 b[31]，模块采用了特殊的减法处理逻辑以符合补码乘法规则；随后，这些部分积进入一个深度为五级的流水线加法树，通过逐级成对相加的方式将数据规模从 32 路压缩至最终的 1 路输出。这种设计不仅通过流水线寄存器提高了电路的最高工作频率，还利用硬件逻辑严谨地处理了补码运算中的正负号扩展问题，确保了在高速时钟下依然能得到正确的 64 位有符号乘法结果。

```

module MULT (
    input  clk,
    input  reset,
    input  signed [31:0] a,
    input  signed [31:0] b,
    output [63:0] z
);
    reg signed [63:0] pipe1 [31:0];
    reg signed [63:0] pipe2 [15:0];
    reg signed [63:0] pipe3 [7:0];
    reg signed [63:0] pipe4 [3:0];
    reg signed [63:0] pipe5 [1:0];
    reg signed [63:0] pipe6;
    integer i;
    always @(posedge clk) begin
        if (reset) begin
            for (i = 0; i < 32; i = i + 1) pipe1[i] <= 64'b0;
            for (i = 0; i < 16; i = i + 1) pipe2[i] <= 64'b0;
            for (i = 0; i < 8; i = i + 1) pipe3[i] <= 64'b0;
            for (i = 0; i < 4; i = i + 1) pipe4[i] <= 64'b0;
            for (i = 0; i < 2; i = i + 1) pipe5[i] <= 64'b0;

```

```

        pipe6 <= 64'b0;
    end else begin
        for (i = 0; i < 31; i = i + 1) begin
            pipe1[i] <= b[i] ? ($signed({{32{a[31]}}}, a)) << i) : 64'b0;
        end
        pipe1[31] <= b[31] ? -($signed({{32{a[31]}}}, a)) << 31) : 64'b0;
        for (i = 0; i < 16; i = i + 1) pipe2[i] <= pipe1[2*i] + pipe1[2*i+1];
        for (i = 0; i < 8; i = i + 1) pipe3[i] <= pipe2[2*i] + pipe2[2*i+1];
        for (i = 0; i < 4; i = i + 1) pipe4[i] <= pipe3[2*i] + pipe3[2*i+1];
        for (i = 0; i < 2; i = i + 1) pipe5[i] <= pipe4[2*i] + pipe4[2*i+1];
        pipe6 <= pipe5[0] + pipe5[1];
    end
end
assign z = pipe6;
endmodule

```

四、测试模块建模

1. 无符号乘法器测试模块

这个 MULTU_tb 模块是一个用于验证乘法器功能的 Verilog 测试平台 (Testbench)，它通过模拟真实的硬件运行环境来自动检测被测设计 (UUT) 的正确性。在具体实现逻辑上，模块首先实例化了待测的乘法器并定义了所需的时钟、复位及数据信号，利用 always 语句产生周期性的时钟脉冲，并在 initial 块中规划了完整的仿真流程：从初始的复位清零开始，在释放复位信号并经过稳定周期后，通过系统函数连续产生五组随机测试向量输入到乘法器中，最后设定了一段较长的等待时间以确保流水线中的所有运算结果都能正确穿透各级寄存器并最终输出。这种测试方式能够高效地模拟连续数据流输入场景，是验证具有时钟延迟特征的流水线硬件模块性能和逻辑准确性的常用手段。

```

module MULTU_tb;

    reg        clk;
    reg        reset;
    reg [31:0] a;
    reg [31:0] b;
    wire [63:0] z;
    MULTU uut (
        .clk (clk),
        .reset(reset),
        .a    (a),
        .b    (b),
        .z    (z)
    );
    always #5 clk = ~clk;
    initial begin

```

```

        clk    = 0;
        reset = 1;
        a      = 0;
        b      = 0;
        #12 reset = 0;
        // 等待复位释放后的稳定周期
        repeat (2) @(posedge clk);
        // 给几组随机数
        @(posedge clk) begin a = $random; b = $random; end
        @(posedge clk) begin a = $random; b = $random; end
        @(posedge clk) begin a = $random; b = $random; end
        @(posedge clk) begin a = $random; b = $random; end
        @(posedge clk) begin a = $random; b = $random; end
        // 等待结果稳定
        repeat (20) @(posedge clk);
    end

endmodule

```

2. 有符号乘法器测试模块

这个 MULT_tb 模块是一个专门针对有符号乘法器设计的仿真测试平台，其核心逻辑是在时域上构建出一套覆盖常规随机值与极限边界值的验证环境。在具体实现上，模块首先配置了时钟振荡电路并完成了待测设计（UUT）的端口映射，通过 initial 初始化序列精确控制复位释放时机，随后不仅利用随机函数连续注入多组有符号测试向量以模拟动态运算场景，还特意针对 32 位有符号数的正负最大值（如 7FFFFFFF 和 80000000）设计了边界压力测试，用以检测补码运算在溢出边缘的逻辑正确性。这种逻辑安排通过流水线式的连续赋值，配合仿真末尾充足的等待周期，能够全面观察数据在穿透硬件流水层级后的稳定输出结果，确保了有符号乘法模块在各种极端数值组合下的运算鲁棒性。

```

module MULT_tb;

    reg          clk;
    reg          reset;
    reg signed [31:0] a;
    reg signed [31:0] b;
    wire signed [63:0] z;

    MULT uut (
        .clk (clk),
        .reset(reset),
        .a    (a),
        .b    (b),
        .z    (z)
    );

    always #5 clk = ~clk;

```

```

initial begin
    clk  = 0;
    reset = 1;
    a    = 0;
    b    = 0;
    #12 reset = 0;
    repeat (2) @(posedge clk);
    @(posedge clk) begin a = $signed($random); b = $signed($random); end
    @(posedge clk) begin a = $signed($random); b = $signed($random); end
    @(posedge clk) begin a = $signed($random); b = $signed($random); end
    @(posedge clk) begin a = $signed($random); b = $signed($random); end
    @(posedge clk) begin a = $signed($random); b = $signed($random); end

    @(posedge clk) begin a = 32'sh7FFFFFFF; b = 32'sh7FFFFFFF; end
    @(posedge clk) begin a = -32'sh80000000; b = -32'sh80000000; end
    @(posedge clk) begin a = 32'sh7FFFFFFF; b = -32'sh80000000; end

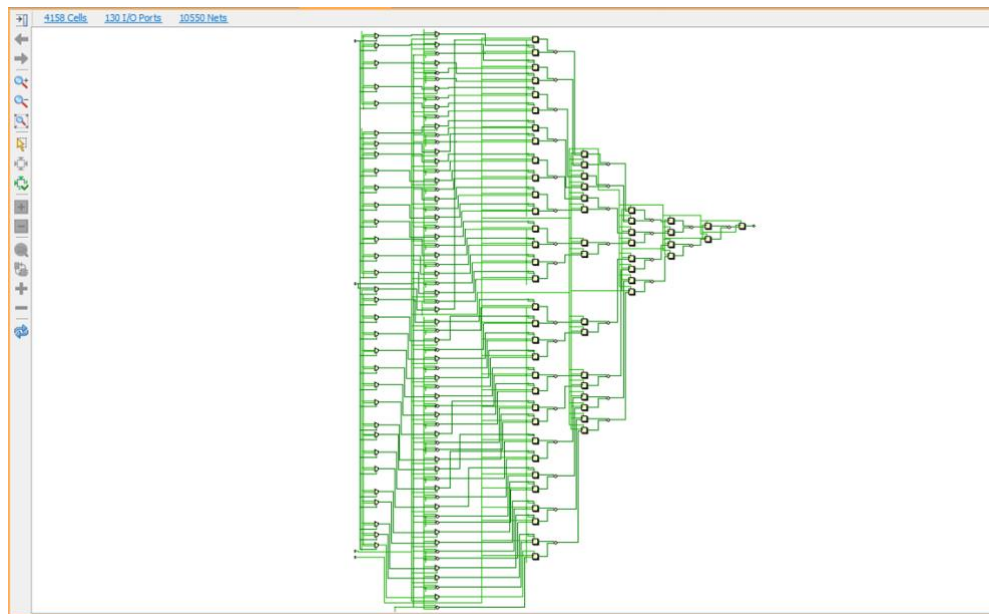
    repeat (20) @(posedge clk);
end

endmodule

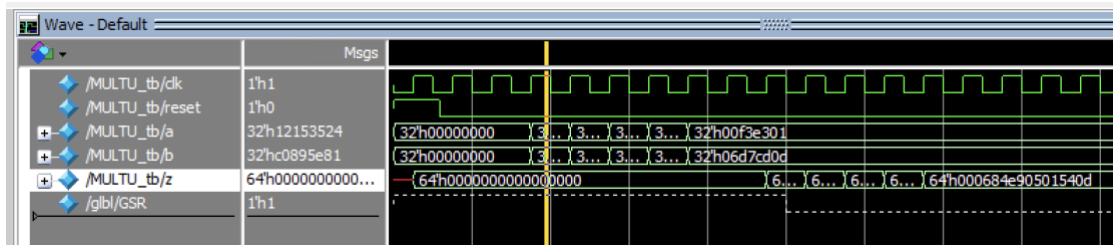
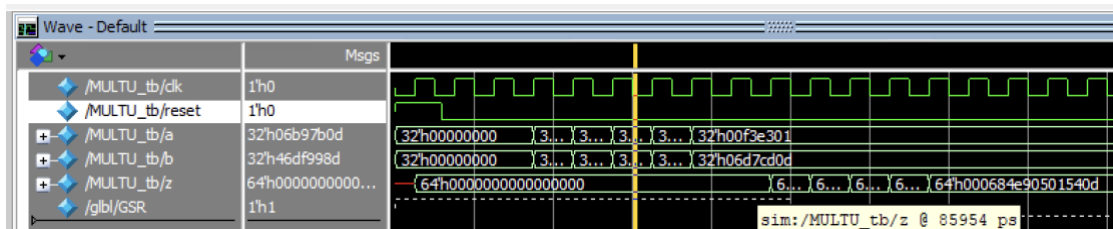
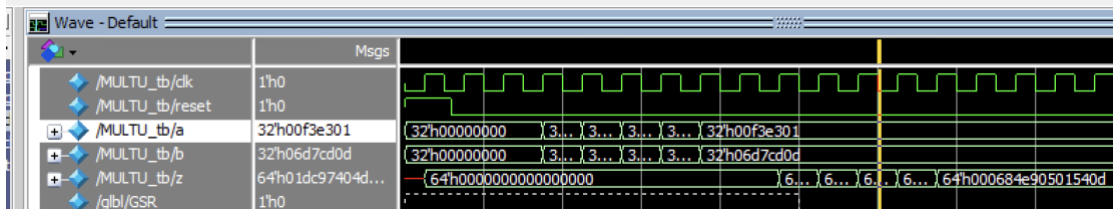
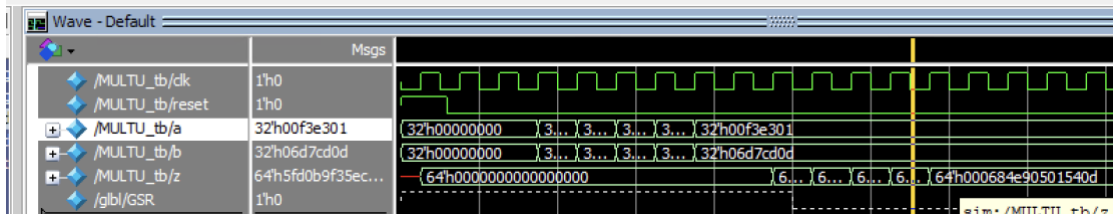
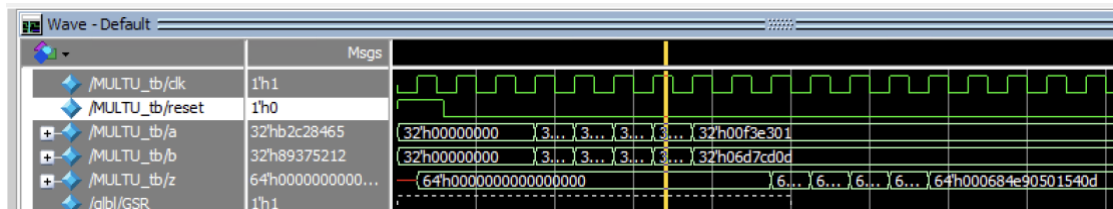
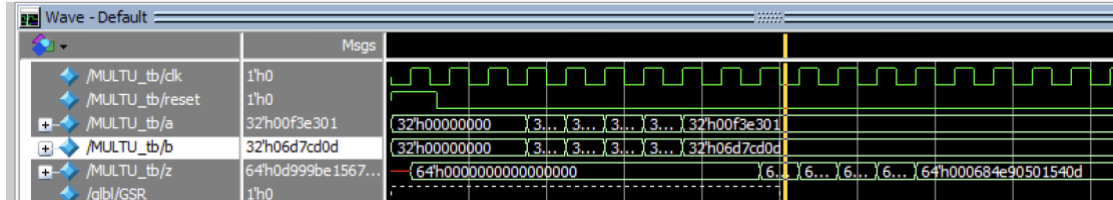
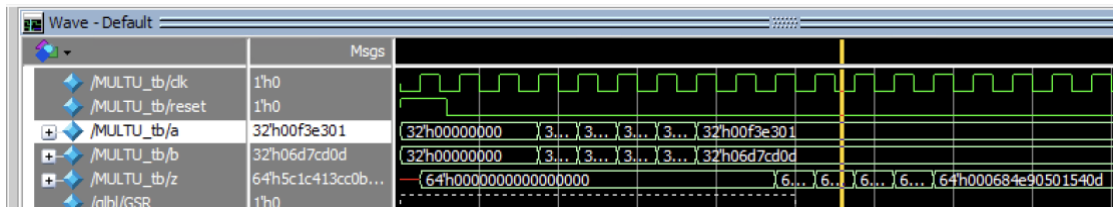
```

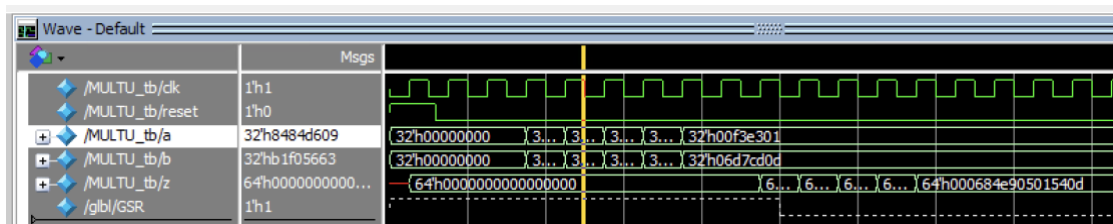
五、实验结果

1. 无符号



下列图经计算可验证正确性





2. 有符号

